AutoSquad

AutoSquad - Fabián Andrés Souto
2º DAM curso 2025/2026

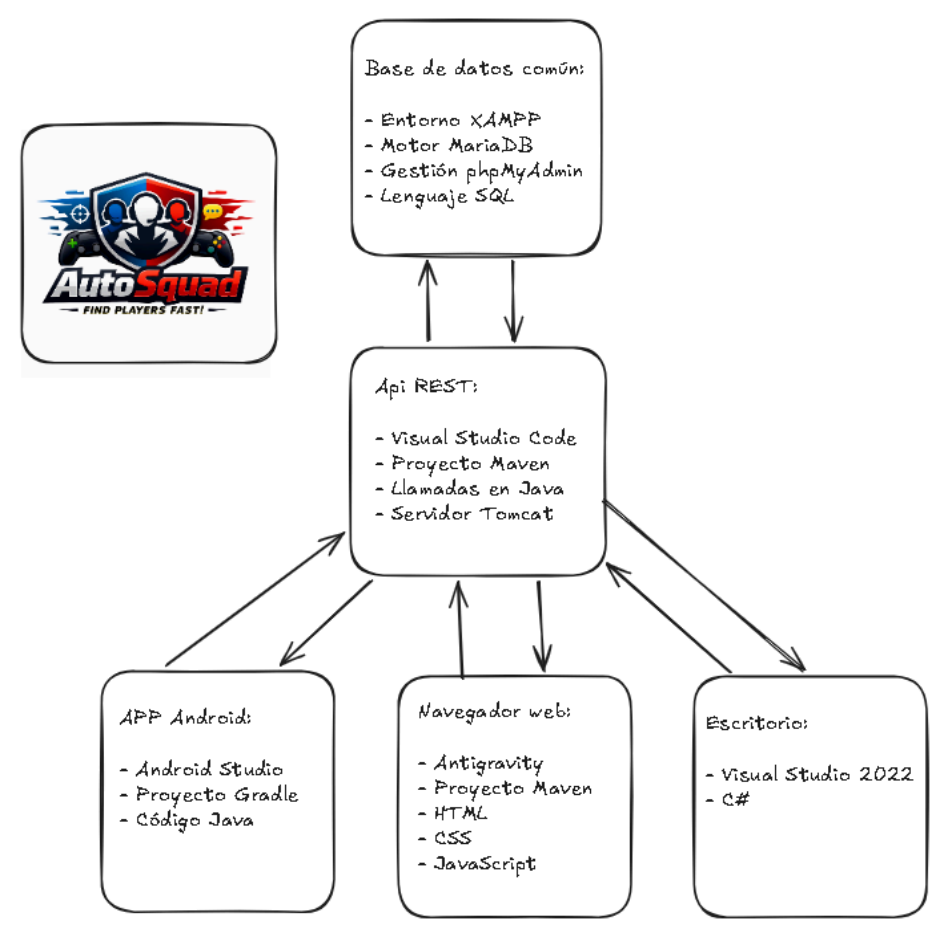
Índice de contenidos

- Arquitectura del sistema
- API
- Base de datos
- Aplicación Web
- Aplicación Móvil
- Aplicación de Escritorio
- Seguridad
- Despliegue
- Guía de uso y mantenimiento
- Mejoras futuras

1. Arquitectura del sistema

El objetivo de nuestro programa es facilitar comunicarse entre jugadores que busquen jugar en equipo pero necesiten nuevos miembros y los necesiten cuanto antes. AutoSquad está compuesto por 3 aplicaciones conectadas (se comunican) con una API REST y ésta a su vez lo hace con la base de datos. Todo funcionará en conjunto para que cada usuario encuentre su compañero ideal lo antes posible.

- Tecnologías principales:
- API: Java
- Base de datos: MySQL
- App móvil: Android (Java)
- App web: HTML, CSS, JavaScript
- App escritorio: C#



2. API

Endpoint	Método	Descripción
/login	POST	Iniciar sesión
/register	POST	Registrar usuario
/caja/create	POST	Crear caja
/caja/list	GET	Listar cajas
/mensaje/create	POST	Enviar mensajes
/mensaje/list	GET	Obtener mensajes

Autenticación:

- Actualmente usa usuario y contraseña y aquí tienes un ejemplo de petición y respuesta para login:

```
String correo = req.getParameter("correo");
String contra = req.getParameter("contra");

PreparedStatement ps = conn.prepareStatement(
        "SELECT id_usuario, nombre, contra FROM USUARIO WHERE correo=?");

ps.setString(1, correo);
ResultSet rs = ps.executeQuery();

if (rs.next() && contra.equals(rs.getString("contra"))) {

   out.print("{\"success\":true," +
           "\"id\":" + rs.getInt("id_usuario") + "," +
           "\"nombre\":\"" + rs.getString("nombre") + "\"}");
} else {
   out.print("{\"success\":false,\"error\":\"Credenciales incorrectas\"}");
}
```

Esa parte de mi API devuelve un JSON indicando si es correcto o incorrecto junto con los datos:

- Resultado correcto:

{
"success": true,
"id": 1,
"nombre": "Fabian"
}

- Resultado incorrecto:

{
"success": false,
"error": "Credenciales incorrectas"
}

3. Base de datos

Tablas:

- Usuario (datos de los diferentes usuarios):

id_usuario	nombre	correo	contra	modos	rango

  - id_usuario: id único
  - nombre: nombre de la cuenta
  - correo: método de inicio de sesión/ contacto del usuario
  - contra: contraseña
  - modos: (futura implementación) especialidades del usuario
  - rango: (futura implementación) estadísticas del usuario

- Caja (cada chat creado):

id_caja	id_creador	id_juego	visibilidad	requisitos	jugadores_buscados	hora_creacion

  - id_caja: identificador de la caja
  - id_creador: identificador del creador
  - id_juego: juego para el que se buscan jugadores

- visibilidad: (futura implementación) indica si se enviará o no según el JSON
- requisitos: especificaciones del usuario
- jugadores_buscados: número de jugadores que queremos
- hora_creación: hora en la que se créa, se utilizará para determinar su vida útil

- Mensaje (datos de cada mensaje enviado):

id_mensaje	id_usuario	id_caja	texto	fecha_hora

- id_mensaje: id único para el mensaje
- id_usuario: usuario que lo crea
- id_caja: caja a la que pertenece
- texto: texto del mensaje
- fecha_hora: hora de creación (se usa para mostrarlo en orden)

- Juego (juegos disponibles actualmente):

id_juego	nombre
1	Valorant
2	LoL
3	CS2

- id_juego: id que indica que juegos soporta la app
- nombre: nombre del juego

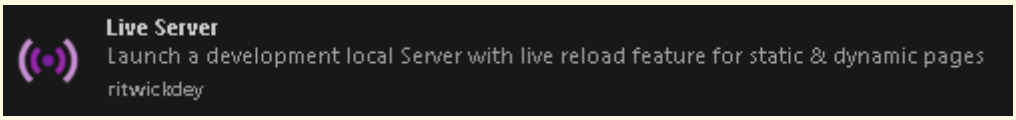
Reglas importantes.

- Cada caja debe existir por una hora para los usuarios (luego se ocultará pero nunca borrará)
- Los usuarios que se encuentren chateando lo podrán seguir haciendo transcurrida la hora (no se interrumpe la conexión)

4. Aplicación Web

- Aplicación de desarrollo: Antigravity
- Lenguajes: HTML, CSS, JavaScript

- Estructura: cada archivo de código HTML contiene sus propios CSS y JS independientes incluidos en el mismo, a futuro se planéa separarlos pero hacer de esta forma evitamos muchos problemas y confusiones que trae el separar los CSS y JS en otro archivo común a todos
- El proyecto es un maven y el programa se puede probar fácilmente con la extensión “Liver Server”:



- La aplicación cliente “consume” la API mediante peticiones HTTP usando fetch, enviando y recibiendo datos en formato JSON, ejemplo:

```
const response = await fetch(`${API_BASE}/login`, {
   method: "POST",
   body: formData
});
```

5. Aplicación Móvil

- Aplicación de desarrollo: Android Studio
- Lenguajes: Java
- Estructura: Actividades, Modelos y Adaptadores
- La aplicación cliente “consume” la API mediante peticiones HTTP usando fetch, enviando y recibiendo datos en formato JSON

6. Aplicación de Escritorio

- Aplicación de desarrollo: Visual Studio 2022
- Proyecto de Windows Forms (.NET Framework)
- Lenguajes: C#
- Estructura: Formularios, Modelos y Servicios

- La aplicación cliente (Form) llama a “ApiService”, usando GET/POST en el endpoint obtenemos los JSON y finalmente los transformamos en los diferentes mensajes, cajas y demás elementos que necesitamos

7. Seguridad

En la aplicación AutoSquad, la seguridad actual se basa en un login básico y el manejo de sesión mediante variables locales, sin uso de tokens ni cifrado de contraseñas, lo que implica ciertos riesgos pero planeamos mejorarlo en las primeras versiones ya que de momento es una versión para ver la recepción del público. Nuestra api y cada programa se encargan de hacer una supervisión básica de los valores introducidos para que la información tenga un formato adecuado y evitar errores.

8. Despliegue

- Backend actual en servidor local (localhost:8080)
- App Android disponible para emulador o móvil (podible descarga desde la PlayStore)
- App web disponible para navegador, se deberá indexar en navegadores a futuro
- App de escritorio, se permitirá descargar la app desde la web de AutoSquad.com
- Desarrollo (ordenador local)
- Para salir a mercado se buscará un hosting para almacenar el servidor o se montará un servidor en casa con posibilidad de redundancia colocando otro en otro lugar

9. Guía de uso y mantenimiento

1. Crear la base de datos en XAMPP (usar archivo “ServidorTablas.txt”) y encenderla (Start: Apache y Start:: MySQL)
2. Encender tomcat con la api (“.war” situado en el proyecto “autosquad-api”)

3. Con la extensión “Live Server” se podrá acceder a la app web del proyecto “autosquad-web” pero se podría subir el proyecto a un servidor privado gratuito estilo glitch.com
4. En móviles tendremos simplemente que entrar a la app y ya funcionará directamente, fué desarrollada en AndroidStudio
5. En la app de escritorio Windows exactamente igual, entramos y funcionará directamente, hemos empleado VS 2022

10. Mejoras futuras

- Se intentará optimizar el código en aspectos como reducir el css o separarlo en archivos
- Se implementarán más juegos como el Fortnite, ClashRoyale, Overwatch2 y Minecraft
- Se incorporará un sistema de pago que permita cosméticos y decoración para el perfil o poder escribir en otros colores y fotos de perfil animadas (gifs)
- Se incorporará una IA para el control de mensajes peligrosos
- Restricciones de edad y verificación para acceder a ciertos juegos
- Se cifrarán mejor las contraseñas y se implementará verificación en 2 pasos o con biometría para mayor seguridad
- Posibilidad de hacer grupos permanentes o más duraderos para creadores de contenido
- Como estamos en servidores locales usamos la ip de nuestra red local pero a futuro deberíamos alquilar un ip para la empresa y así poder indexarnos y permitir a nuestras aplicaciones tener cierta homogeneidad